

The Sovereign Substrate

A Machine-Checked Model of Constitutional Currency

S. Wheeler

v2.0

Formal companion to *The Sovereign Substrate: A constitutional architecture for governed computation* (S. Wheeler, v2.0, 2026; <https://doi.org/10.5281/zenodo.19960841>).

Correspondence: swheeler-research@proton.me

Paper repository: <https://github.com/swheeler-research/substrate-research>

Reference implementation: <https://github.com/swheeler-research/substrate-reference>

Licence: CC BY 4.0

Contents

Abstract	3
1. The two questions	3
2. The faithful encoding, and why the natural one is wrong	4
3. The bounded-latency theorem	5
4. What reduces to standard results, named as such	7
5. Traceability from architecture to model	7
6. What this paper is	8
Appendix. Artifacts	9

Abstract

The Sovereign Substrate lists, among the formal specifications it identifies as open, two that concern constitutional currency: a specification of compile-at-commit semantics under which “no act executes under stale governance” can be proved, and a specification of the uniform invalidation surface under which “compiled forms cease to authorise execution within bounded latency after invalidation events” can be proved. This paper supplies both, for a model of the surface, and reports the results and their limits plainly.

Two things are established. First, the faithful formalisation of “no act executes under stale governance” is a property of invocation events, not a state invariant over which compiled forms are invocable. The natural state-invariant encoding, that an invocable form is never affected by a pending governance change, is false, and a model checker produces the counterexample automatically: in the adoption interval between a governing change and the invalidation of a dependent form, the form is affected and still marked invocable. That interval is the adoption latency the substrate admits in its own statement of the property, not a defect, and the correct encoding follows the substrate’s own language: the act refuses at the moment of invocation rather than the form being unaffected at all times. Second, the bounded-latency claim holds as a single certified theorem: a form affected by any of the six triggers becomes non-invocable no later than its deadline, fixed at the first triggering event to affect it plus a bound Δ , over a governance clock the invalidation surface disciplines rather than an unconstrained one. The bound is certified by the TLA+ Proof System for arbitrary instance cardinality, over a model in which the constitutional-source update propagates through a roots relation pinned to the transitive closure of the credential-derivation graph for an acyclic parent relation, with that cascade independently model-checked to multi-level depth.

The paper corroborates the substrate’s stated semantics while refuting a natural but incorrect state-based formalisation of those semantics. The mechanism the substrate describes, uniform per-act refusal of invalidated forms with an admitted, bounded adoption latency, is coherent and is what the model exhibits; what the paper corrects is not the architecture but the encoding a formaliser would reach for first, which the architecture’s own admission of adoption latency already tells against. These are results about a model; reading them as the architecture’s own properties rests on a stated and traceable rendering of the substrate, given in section 5, which also names the modelling judgements and scope limits a reader may dispute. What remains open is what the substrate already says is open: a timing model relating the governance clock to wall-clock latency, and the engineering cost of the recompilation cascade at scale.

1. The two questions

The substrate names the formal work it would value, and two of its questions concern currency directly. The first asks for a specification of compile-at-commit semantics under which the property “no act executes under stale governance” can be proved. The second asks for a specification of the uniform invalidation surface under which “compiled forms cease to authorise execution within bounded latency after invalidation events” can be proved. This paper answers both, for a model of the surface.

The substrate already states the property with care, and the care is what makes it provable. It admits adoption latency, the interval between a governance commitment and the propagation of that commitment into compiled forms, and it refuses enforcement latency: during the adoption interval, a stale compiled form produces refusal rather than executing under stale governance. So “no act executes under stale governance” claims that an act either runs under current governance or refuses, and never executes under governance that has changed; it does not claim that governance changes take effect instantaneously. The first question is about that refusal property; the second is about the bound on how long the adoption interval can last. The paper takes them in turn.

2. The faithful encoding, and why the natural one is wrong

A formaliser asked to encode “no act executes under stale governance” reaches first for a state invariant over compiled forms: for every form, if it is invocable then none of its governing inputs has changed. Written over the substrate’s state, this reads $\text{invocable}[f] \Rightarrow \neg \text{Affected}(f)$, where $\text{Affected}(f)$ holds when some governing input of f has changed. This is the encoding to avoid.

It is false, and a model checker shows it. A governing change, a credential revocation, a supersession, a deprecation, a constitutional-source update, a drift detection, an integrity failure, occurs as an event, and the invalidation surface marks the dependent forms in response. Between the change and the marking there is an interval in which a form is affected and still carries its invocable mark. The state invariant therefore has a reachable counterexample one step after any governing change. TLC, checking a bounded instance over its complete state graph, reports the violation and prints the trace: an initial state, then a credential revocation, producing a state in which the credential is revoked while both dependent forms remain invocable with deadlines set (MC.tla, MC.cfg, configuration MCnaive.cfg, transcript tlc_naive_refutation_output.txt; a separate enumeration check_model.py, written against the same model, reproduces it). The counterexample is generated by the checker’s search, not constructed by hand, which is the evidence that the encoding’s failure is structural.

The failure is the encoding’s, not the substrate’s, and the reason is the adoption latency the substrate admits. The substrate never claims $\text{invocable}[f] \Rightarrow \neg \text{Affected}(f)$; it claims that the act, at the moment of invocation, refuses if the form has been invalidated. The faithful encoding is therefore a property of invocation events. The model represents invocation by a guarded action $\text{InvokeStrict}(f)$ that may fire only when $\neg \text{Affected}(f)$, so every committed invocation occurs in an unaffected state, and an affected form’s invocation refuses. The property holds of the model by construction, since InvokeStrict is guarded by $\neg \text{Affected}(f)$ and no action rewrites a committed ledger entry. It is recorded rather than checked: the finite-instance model MC.tla carries a flag whose only writer assigns a conjunct of that guard, so the corresponding invariant is a tautology, and the ledger formulation Inv_tagged in $\text{InvalidationSurface.tla}$ is stated but not model-checked. The property is immediate once the guard is written, and is not offered as a difficult theorem; its role is to be the correct statement of the substrate’s claim, against which the tempting wrong encoding can be measured. The content of this section is the relocation: currency is enforced at the invocation event, where the runtime re-checks the form’s validity, and the invalidation surface supplies the bounded cleanup that the next section bounds.

The relocation matches the substrate’s own account, in which the runtime invokes only against unexpired compiled forms and stale forms refuse during the adoption interval. The formal contribution is to identify that the act-event reading, not the state-invariant reading, is the one that is both true and faithful to the prose, and to show mechanically why the alternative a formaliser would reach for first cannot be sustained.

3. The bounded-latency theorem

The second question asks for a bound: after an invalidation event, a compiled form ceases to authorise execution within bounded latency. The substrate’s own answer is that recompilation latency is bounded by the compilation pipeline’s operational characteristics, in time scales operators specify in their authored content as conditions of conforming operation. Delta renders that operator-declared parameter. This section certifies a model in which that bound holds, as a single theorem, uniformly across all six triggers.

A TLA+ model (`InvSurface_transitive_cascade_tlaps.tla`) represents each trigger as a distinct action that records a governance verdict, and arms a deadline on each newly affected form at the triggering time plus Delta. The six triggers reduce to three structural shapes, and the paper states this rather than implying six independent mechanisms: revocation, supersession, deprecation, and drift detection each add to a monotone set and affect a form whose chain or unit lies in that set; integrity failure sets a per-form flag; the constitutional-source update is the one trigger whose effect reaches a form by propagation rather than direct membership, affecting a form when some credential in its chain has an updated source among its transitive roots. The roots relation is constrained by the fixpoint property that a source is a transitive root of a credential exactly when the credential is that source or some immediate parent has it as a transitive root. The property pins roots to the transitive closure when the parent relation is acyclic, which the substrate gives by specifying the credential graph as acyclic rather than by freezing it at compile time; under a cyclic parent relation it admits relations that over-approximate the closure. The constraint fixes the cascade’s meaning in the model and is what makes the model-checked witness a multi-level one. The constraint appears in the USE clause of the proofs of the theorems the paper names, so it is available to every step of those proofs. Whether any step depends on it is not established here: the bounded-response argument as written needs only that roots is a constant of the declared type and that the set of updated sources grows, and no variant module is shipped that discharges the obligations without the constraint. Establishing the independence would take one run of the prover against such a variant, and that run has not been made. Nor is the transitivity checked: the model-checked instance supplies a roots relation directly, and its agreement with that instance’s parent relation is asserted in the module’s comments. The constraint therefore fixes what the cascade means in the model and is corroborated by neither the proof nor the model check. The substrate states this trigger as direct membership, since its resolved authority chain already terminates at the constitutional source; the model represents the chain as the credentials named directly and recovers the closure through a separate roots relation, which keeps the propagation visible and is the reason this trigger is the one checked at depth.

A clock action advances governance time and is guarded so it cannot pass the deadline of any form that is still affected and invocable. That guard carries the

safety invariant's clock case, and SpecL imposes no fairness on the clock action, so the model admits behaviours in which the governance clock never advances at all. The reading that survives is therefore of a clock the invalidation surface disciplines, and the architecture's own governance clock advances once per act, so it is not a clock an implementation is free to hold back until a cascade settles. What the theorem establishes is a property of the model's clock, and relating it to a clock an implementation would run is work this development does not do. Over this model `tlapm` certifies the bounded-response theorem for arbitrary instance cardinality (transcript `tlaps_transitive_cascade_output.txt`, all 301 obligations proved: 199 recorded as trivial and closed by `tlapm` without a backend, 96 by the SMT backend, and six by the temporal backend LS4; five of the backend obligations were served from the prover's fingerprint cache rather than discharged in that run):

$$\text{SpecL} \Rightarrow \forall f \in \text{Form} : (\text{Affected}(f) \wedge \text{invocable}[f]) \rightsquigarrow (\neg \text{invocable}[f] \wedge \text{now} \leq \text{deadline}[f])$$

Affected is the disjunction of all six triggers, so the one theorem covers each by the same argument. Because the deadline is fixed at the triggering time plus `Delta`, the target of the leads-to carries the bound: a form affected by any trigger becomes non-invocable at a governance-clock time no later than `triggerTime + Delta`, where the clock is one that may not advance past a pending deadline. The theorem is therefore a property of a governance clock disciplined by the invalidation surface, not a bound an undisciplined clock would respect. The proof is the weak-fairness rule WF1 with its target strengthened to carry the bound; the certified clock-advancement invariant `BoundInv` (theorem `BoundSafety`) gives $\text{now} \leq \text{deadline}[f]$ while the form is stale and invocable, and the only transition out of invocability, the invalidation action, alters neither the clock nor the deadline, so the bound holds at the moment of enforcement. The safety invariant and the clock-advancement invariant are certified over the same model, so it is sound at the safety level and the bounded-response level together. The fairness assumption is named in the antecedent `SpecL`: progress requires that invalidation eventually fires, and the bound on the clock once it does is unconditional.

The constitutional cascade is the one trigger whose bound is not immediate by membership, so it is checked independently and at depth. A TLC check on a bounded instance with a two-level credential chain, a constitutional source and a credential derived from it, confirms the safety invariant over the complete state space reachable within the clock bound $\text{now} \leq 3$, the constraint that makes the space finite, with no violation (transcript `tlc_transitive_cascade_output.txt`; the state count is recorded in the appendix). On the same instance, a non-vacuity check exhibits a reachable trace in which a source update affects a form whose chain contains only the derived credential, not the source itself, because the source is among that credential's transitive roots, and the form is then invalidated (transcript `tlc_transitive_cascade_depth2_output.txt`). The cascade propagates through an intermediate credential the form does not directly contain; the theorem holds for it and it is not secretly one-step.

One note on reading the theorem. Its target $\neg \text{invocable}[f] \wedge \text{now} \leq \text{deadline}[f]$ is a state predicate; the latency reading, that enforcement occurs no later than the deadline, follows because the governance clock is monotone non-decreasing, the only action that changes *now* being `Tick`, which sets $\text{now}' = \text{now} + 1$, so the

enforcement instant is at clock time at most the deadline. The bound is on the governance clock, the logical tick the substrate's recompilation cascade is measured in, not wall-clock time; relating the two would need a timing model this development does not provide, and the substrate names that same gap.

4. What reduces to standard results, named as such

Two adjacent results are included for completeness and are not contributions. The cross-operator case, where one operator runs forms whose authority derives from another it observes with a lag, reduces to standard bounded-staleness reasoning: stale invocations are bounded by the dependent operator's invocation rate times the propagation lag, and under partition the safe behaviour is refusal. This is the application of known recency constraints from the replication literature to governance authority, sound under consistent-prefix delivery and monotonic reads, and is not novel. The staged-operation case yields a design rule rather than a theorem: an externally visible effect committed through a governed unit should occur only at a state from which the operation may permanently stop, so that a halt does not strand the effect. A bounded enumeration (`check_seam.py`) confirms the rule is non-vacuous and identifies operations for which no safe in-order staging exists; a theorem-level treatment is future work.

5. Traceability from architecture to model

These are results about a model, so the burden is to show the model renders the architecture faithfully. The mapping is given explicitly, so a reader can check each element against the substrate text rather than trust the translation.

Substrate mechanism	Model element
Credential revocation	<code>RevokeCred(c)</code> : add <code>c</code> to <code>crevoked</code> ; affects forms with <code>c</code> in chain
Credential supersession	<code>SupersedeCred(c)</code> : add <code>c</code> to <code>csuperseded</code> ; affects forms with <code>c</code> in chain
Credential or unit deprecation	<code>Deprecate(x)</code> : add <code>x</code> to <code>cdeprecated</code> ; affects forms with <code>x</code> in chain or as unit
Constitutional-source update	<code>UpdateSource(s)</code> : affects forms with a chain credential <code>c</code> where <code>s \in roots[c]</code> , <code>roots</code> a fixpoint of the parent relation, the transitive closure when the relation is acyclic
Drift detection	<code>DriftUnit(u)</code> : add <code>u</code> to <code>drifted</code> ; the marking event, not the divergence
Compilation integrity failure	<code>FailIntegrity(f)</code> : set <code>integrity[f]</code> to failed
Refuse invalidated forms at the act	<code>Invalidate(f)</code> removing invocability; <code>InvokeStrict</code> guarded by <code>~Affected</code> , a stuttering step in the headline module, and <code>Invoke_strict</code> recording to the ledger in <code>InvalidationSurface.tla</code>
Bounded adoption latency	<code>Delta</code> , the deadline, and the clock guard

The six triggers are taken from the substrate’s own specification of the invalidation surface. The modelling judgements that carry weight are these. The authority chain is fixed per form, faithful to the substrate’s chain frozen at compile time but not modelling recompilation changing a form’s chain. The cascade’s roots relation is constrained by a fixpoint characterisation rather than computed from a concrete graph, which is sound because the substrate freezes that graph at compile time. Nothing in the model confines the constitutional sources to the credential set, so the theorem also holds over instances in which the cascade disjunct can never be satisfied. Drift is modelled as the marking event, with detection latency, the time from a unit beginning to drift to the windowed observations crossing the band, outside scope; the bound is on enforcement after detection. Trigger sets grow until a form is invalidated, so withdrawal of a deprecation and reset of drift are not modelled; this is sound for the bound, since un-affecting a form cannot extend the interval the bound governs. The omission is not minor in architectural terms: the substrate requires a conforming implementation to support withdrawal of a deprecation and reset of a drift state, so the model’s monotone trigger sets exclude mechanisms the architecture mandates, and with them the model’s own recompilation path. And the bound is measured from the substrate event that records a trigger, not from a backdated effective time, and assumes non-degraded operation; the substrate admits both backdated effective times and degraded modes, each of which is a staleness consideration outside this model. The bound is assumed positive, which excludes a zero-latency invalidation surface from the model. Supersession is modelled over credentials, which is what the substrate specifies; a replaced unit is a new unit, and its predecessor reaches a form through the deprecation trigger. Deprecation is modelled as unconditional, where the substrate admits an operator-authored exception permitting composition against deprecated units. And the model carries a recompilation action that no reachable state enables, because the trigger sets grow monotonically, so a form once affected stays affected; the action is inert rather than load-bearing, and the bound does not rest on it. A reader who disputes any of these disputes the model, not the proof; the proof is sound for the model as given.

6. What this paper is

The paper answers two formal questions the substrate poses about constitutional currency, for a model of the invalidation surface. It establishes that the faithful encoding of “no act executes under stale governance” is an invocation-event property and that the natural state-invariant encoding is false, with the falsity machine-exhibited and traceable to the adoption latency the substrate itself admits. And it certifies the bounded-latency claim as a single theorem, uniform across all six triggers, over a model in which the constitutional cascade is resolved through a roots relation and independently model-checked to depth.

The result corroborates the substrate. The mechanism the substrate describes, uniform per-act refusal of invalidated forms with an admitted and bounded adoption latency, is coherent, and the model exhibits it; the bounded-latency property the substrate states informally is here a certified theorem of that model. The paper makes no claim that the architecture is wrong, so nothing in the substrate must change for the paper to stand. What remains open is what the substrate already names as open: a timing model relating the governance clock to wall-clock latency, and the engineering cost of the recompilation cascade at scale, which an implementation would have to measure. The work that would most strengthen the paper is

co-development rather than further proof: a revised substrate runtime section precise enough that the model’s terms attach to its clauses directly, at which point the theorems become statements about the architecture as specified rather than about a faithful rendering of its prose.

Appendix. Artifacts

The formulas above use standard mathematical notation; the accompanying TLA+ modules use TLA+’s ASCII syntax, in which $\forall$ is the universal quantifier, $\wedge$ is conjunction, $\sim$ is leads-to, and $\neg$ is negation.

The headline development certifies the bounded-response theorem over all six triggers:

- `InvSurface_transitive_cascade_tlaps.tla`, checked by `tlapm` (transcript `tlaps_transitive_cascade_output.txt`, all 301 obligations proved: 199 by `tlapm`’s own reasoning, 96 by the SMT backend, and six by LS4). It certifies the clock-advancement invariant `BoundInv` through theorem `BoundSafety`, the weakening `NoOvershoot` that reads the bound off it, and the bounded-response theorem `LiveB`, with the constitutional cascade resolved through a roots relation constrained by a fixpoint property over the credential parent relation.
- `SixTransCheck.tla` with `MCtrans.tla` and `MCtrans.cfg`: the TLC cross-check on a two-level credential chain, safety invariant holding over the 1,688,888 distinct states reachable under the state constraint $now \leq 3$, with no violation; the transcript is `tlc_transitive_cascade_output.txt`. `SixTransCheck.tla` is the proof module with the proof apparatus and the assumptions removed, since TLC takes the constants from the configuration; the instance’s roots relation is supplied directly and its agreement with the instance’s parent relation is asserted in the module’s comments rather than checked.
- `MCtransNV.cfg` (transcript `tlc_transitive_cascade_depth2_output.txt`): the depth-two cascade witness, a reachable trace in which a source update reaches a form through a derived credential it does not directly contain.

The encoding result rests on the single-operator model:

- `InvalidationSurface.tla`: the single-operator TLA+ module.
- `MC.tla` with `MC.cfg` and `MCnaive.cfg` (transcripts `tlc_invariants_output.txt` and `tlc_naive_refutation_output.txt`): safety holding over the bounded graph, and the naive state invariant violated with the counterexample printed. The second configuration differs from the first only in the invariant it checks. In both, the clock bound that makes the space finite is inside the module’s own clock action rather than in the configuration, which is why neither configuration carries a state constraint.
- `check_model.py` and `check_seam.py`: a second enumeration of the single-operator model, independent of TLC but not of the model, and the seam non-vacuity check.
- `InvSurface_safety_tlaps.tla` (transcript `tlaps_safety_output.txt`, 65 obligations proved: 50 recorded as trivial and closed by `tlapm` without a backend, 14 by the SMT backend, and one by LS4, with all fifteen backend obligations served from the prover’s fingerprint cache rather than discharged in that run): the certified safety proof of the bound’s first half, over a single-operator encoding that tracks live deadlines through a separate function.

- `InvSurface_liveness_tlaps.tla` and `InvSurface_composed_tlaps.tla` (transcripts `tlaps_liveness_output.txt`, `tlaps_composed_output.txt`), with the SMT cross-check `tlaps_z3_proof.py`: the earlier four-trigger lineage that the transitive-cascade model supersedes, retained for reference. The SMT cross-check requires a solver that the build notes do not install and ships no transcript.

The TLAPS proofs were checked with `tlapm` built from source (the TLA+ Proof System development version). Zenon, Z3 and LS4 were assembled as backends; the transcripts record every obligation as discharged by `tlapm`'s own reasoning, by the SMT backend, or by LS4, which is required for the temporal steps. Zenon discharged none of them. The build is not a one-command install; the steps are recorded with the artefacts so the result can be reproduced. The TLC checks were run under TLC 2.15. The bounded checks are what is bounded: they exercise the model and produce the counterexample to the naive invariant on small instances. The TLAPS proofs quantify over the uninterpreted constant sets and so hold at arbitrary instance cardinality, not only the checked instances.